

1. Frontend (Node.js with Express):

- Create a frontend using Express and Node.js.
- Include a form similar to the one from the Flask Assignment 2.
- Configure the form to send a request to the Flask backend.
- **Backend (Flask):**
- Use the Flask backend to handle the form submission and process the data.
- **Folder Structure:**
- Organize the project with separate folders for the frontend and backend.
- **Docker Configuration:**
- Create a `Dockerfile` for both the frontend and backend.
- Write a `.yaml` file (Docker Compose) to connect both services in the same network.
- Upload both images to docker hub and push your whole code to github and add the `node_modules` and other non required files(`.vscode`) in `.gitignore`

Link - <https://github.com/Toto3107/flaskp>

Submission Guidelines -: 1. Upload the terminal screen shots and all your project code or files on github.
2. Share your github repo link

Answer

This looks like the core assignment requirement that sets up your multi-container environment! To adapt this specifically for a **Node.js/Express Frontend** communicating with a **Flask Backend**, here is the structural configuration and the `docker-compose.yml` needed to tie them together in the same network.

1. Updated Project Structure

Ensure your repository includes a `.gitignore` file at the root to exclude heavy or local environment directories like `node_modules`, `__pycache__`, and `.vscode`.

```
flaskp/
├── .gitignore
├── docker-compose.yml
├── frontend/
│   ├── Dockerfile
│   ├── package.json
│   ├── server.js      # Express server serving the form
│   └── views/
│       └── index.html  # Form page
└── backend/
```

```
|— Dockerfile
|— app.py      # Flask processing API
|— requirements.txt
```

Ideal `.gitignore` for this project:

```
node_modules/
__pycache__/
*.pyc
.env
.vscode/
.DS_Store
```

2. Docker Configuration

Frontend `Dockerfile` (`frontend/Dockerfile`)

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Backend `Dockerfile` (`backend/Dockerfile`)

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Multi-Container Orchestration (`docker-compose.yml`)

This Compose file creates a dedicated bridge network named `app-network`. It maps the Express frontend to port `3000` and the Flask backend to port `5000`. By defining an environment variable (`FLASK_BACKEND_URL`), the Express app can easily route form submissions across the network using the backend's service name.

```
version: '3.8'
```

```
services:
```

```
  frontend:
```

```
    build: ./frontend
```

```
    container_name: express_frontend
```

```
    ports:
```

```
      - "3000:3000"
```

```
    environment:
```

```
      - FLASK_BACKEND_URL=http://backend:5000
```

```
    networks:
```

```
      - app-network
```

```
    depends_on:
```

```
      - backend
```

```
  backend:
```

```
    build: ./backend
```

```
    container_name: flask_backend
```

```
    ports:
```

```
      - "5000:5000"
```

```
    networks:
```

```
      - app-network
```

```
networks:
```

```
  app-network:
```

```
    driver: bridge
```

3. Connecting the Frontend to the Backend

To ensure successful communication between the containers, your Express frontend needs to route the form data to the Flask container using its service alias defined in the Compose file.

Express Route Example (`frontend/server.js`)

```
const express = require('express');
```

```

const axios = require('axios'); // Install axios via npm
const app = express();

app.use(express.urlencoded({ extended: true }));
app.use(express.json());

const BACKEND_URL = process.env.FLASK_BACKEND_URL || 'http://localhost:5000';

// Handle form submission
app.post('/submit-form', async (req, res) => {
  try {
    // Forwarding form data to the Flask API
    const response = await axios.post(`${BACKEND_URL}/api/process`, req.body);
    res.status(200).send(`Backend Responded: ${JSON.stringify(response.data)}`);
  } catch (error) {
    res.status(500).send(`Error connecting to Flask backend: ${error.message}`);
  }
});

app.listen(3000, () => console.log('Frontend listening on port 3000'));

```

Flask Endpoint Example ([backend/app.py](#))

```

from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/api/process', methods=['POST'])
def process_data():
    data = request.json or request.form
    # Simple processing logic mimicking your previous assignment
    return jsonify({
        "status": "success",
        "received_data": data,
        "message": "Data processed successfully by Flask backend"
    }), 200

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

4. Execution Commands & Upload Steps

To complete the requirements for Docker Hub and GitHub, execute the following steps in your terminal:

1. Build and run the multi-container setup locally to test
`docker-compose up --build -d`

2. Tag and Push images to Docker Hub

Replace 'your_dockerhub_username' with your actual username

`docker tag flaskp-frontend:latest your_dockerhub_username/express-frontend:latest`

`docker tag flaskp-backend:latest your_dockerhub_username/flask-backend:latest`

`docker push your_dockerhub_username/express-frontend:latest`

`docker push your_dockerhub_username/flask-backend:latest`

3. Commit and push the clean architecture to GitHub

`git add .`

`git commit -m "Feat: Add Express frontend, Flask backend, and Docker Compose setup"`

`git push origin main`